OOP In System Verilog

Introduction

This report documents the verification of an APB slave using a SystemVerilog object-oriented testbench. The environment comprises a generator, driver, monitor, and scoreboard, interconnected via mailboxes and synchronized with semaphores and events. The testbench generates random write/read transactions, drives them onto the APB bus, and compares actual results against a reference model. Simulations confirm correct protocol behavior and data integrity across all transactions, with waveforms presented for analysis.

Code:

Design.sv

```
//=====================================================
// APB INTERFACE (full signal set)
//=====================================================
interface apb_if(input logic clk, input logic rst_n);
    logic        psel;
    logic        penable;
    logic        pwrite;
    logic [7:0]  paddr;
    logic [31:0] pwdata;
    logic [31:0] prdata;
    logic        pready;
    logic        pslverr;
endinterface

//=====================================================
// APB SLAVE DUT
//=====================================================
module apb_slave (
  input  logic        pclk,
  input  logic        presetn,
  input  logic        psel,
  input  logic        penable,
  input  logic        pwrite,
  input  logic [7:0]  paddr,
  input  logic [31:0] pwdata,
  output logic [31:0] prdata,
  output logic        pready,
```

```
  output logic         pslverr
);
  logic [31:0] mem [0:15];
  integer i;

  always_ff @(posedge pclk or negedge presetn) begin
    if (!presetn) begin
      prdata  <= 32'h0;
      pready  <= 1'b0;
      pslverr <= 1'b0;
      for (i = 0; i < 16; i++) mem[i] <= 32'h0;
    end else begin
      pready  <= 1'b0;
      pslverr <= 1'b0;
      if (psel && penable) begin
        pready <= 1'b1;
        if (paddr[1:0] != 2'b00) pslverr <= 1'b1;
        else if (paddr[7:6] != 2'b00) pslverr <= 1'b1;
        else begin
          if (pwrite) mem[paddr[5:2]] <= pwdata;
          else        prdata <= mem[paddr[5:2]];
        end
      end
    end
  end
endmodule
```

Tb.sv

```
//=====================================================
// TRANSACTION
//=====================================================
class transaction;
    rand bit [7:0]  addr;
    rand bit [31:0] wdata;
    bit [31:0]      rdata;
    bit             write;
    bit             read;
    bit             error;

    constraint valid_addr {
        addr[1:0] == 0;
        addr[7:6] == 0;
    }

    function void display(string name);
        $display("[%s] ADDR=%0h %s DATA=%0h", name, addr,
              write ? "WRITE" : "READ ", write ? wdata : rdata);
    endfunction
endclass

//=====================================================
// GENERATOR (5 transactions, @done)
//=====================================================
class generator;
    mailbox gen_drv, gen_scb;
```

```
    event   done;
    int     num_trans = 5;

    function new(mailbox gd, mailbox gs, event d);
        gen_drv = gd;
        gen_scb = gs;
        done    = d;
    endfunction

    task run();
        transaction tr;
        repeat (num_trans) begin
            tr = new();
            assert(tr.randomize()) else $error("Randomization
failed");
            if ($urandom_range(0,1)) begin
                tr.write = 1;
                tr.read  = 0;
            end else begin
                tr.write = 0;
                tr.read  = 1;
                tr.wdata = 0;
            end
            tr.display("GENERATOR");
            gen_drv.put(tr);
            gen_scb.put(tr);
            @(done);   // block until scoreboard triggers
        end
        $display("[GENERATOR] All transactions completed");
    endtask
endclass

//=====================================================
// DRIVER
//=====================================================
class driver;
    virtual apb_if vif;
    mailbox gen_drv;
    semaphore sem;

    function new(virtual apb_if v, mailbox gd, semaphore s);
        vif     = v;
        gen_drv = gd;
        sem     = s;
    endfunction

    task run();
        transaction tr;
        forever begin
            gen_drv.get(tr);
            sem.get(1);
            @(negedge vif.clk);
```

```
            vif.psel    = 1;
            vif.penable = 0;
            vif.pwrite  = tr.write;
            vif.paddr   = tr.addr;
            vif.pwdata  = tr.wdata;
            @(negedge vif.clk);
            vif.penable = 1;
            @(posedge vif.clk);
            while (!vif.pready) @(posedge vif.clk);
            tr.display("DRIVER");
            @(negedge vif.clk);
            vif.psel    = 0;
            vif.penable = 0;
            sem.put(1);
        end
    endtask
endclass

//=============================================================
// MONITOR
//=============================================================
class monitor;
    virtual apb_if vif;
    mailbox mon_scb;

    function new(virtual apb_if v, mailbox ms);
        vif     = v;
        mon_scb = ms;
    endfunction

    task run();
        transaction tr;
        forever begin
            @(posedge vif.clk);
            if (vif.psel && vif.penable && vif.pready) begin
                tr = new();
                tr.addr  = vif.paddr;
                tr.write = vif.pwrite;
                tr.read  = !vif.pwrite;
                tr.error = vif.pslverr;
                if (vif.pwrite)
                    tr.wdata = vif.pwdata;
                else
                    tr.rdata = vif.prdata;
                tr.display("MONITOR");
                mon_scb.put(tr);
            end
        end
    endtask
endclass
```

```
//=============================================================
// SCOREBOARD
//=============================================================
class scoreboard;
    mailbox expected, actual;
    event   done;
    bit [31:0] mem [bit [7:0]];

    function new(mailbox exp, mailbox act, event d);
        expected = exp;
        actual   = act;
        done     = d;
    endfunction

    task run();
        transaction exp, act;
        forever begin
            expected.get(exp);
            actual.get(act);
            if (exp.write) mem[exp.addr] = exp.wdata;
            if (exp.error != act.error) begin
                $display("ERROR mismatch: exp=%b act=%b",
exp.error, act.error);
            end else if (exp.write) begin
                if (exp.addr == act.addr && exp.wdata ==
act.wdata)
                    $display("SCOREBOARD: WRITE PASS (ADDR=%0h
DATA=%0h)", exp.addr, exp.wdata);
                else
                    $display("SCOREBOARD: WRITE FAIL (ADDR
exp=%0h act=%0h, DATA exp=%0h act=%0h)",
                             exp.addr, act.addr, exp.wdata,
act.wdata);
            end else begin
                if (exp.addr == act.addr && mem[exp.addr] ==
act.rdata)
                    $display("SCOREBOARD: READ PASS (ADDR=%0h
DATA=%0h)", exp.addr, act.rdata);
                else
                    $display("SCOREBOARD: READ FAIL (ADDR
exp=%0h act=%0h, DATA exp=%0h act=%0h)",
                             exp.addr, act.addr, mem[exp.addr],
act.rdata);
            end
            -> done;
        end
    endtask
endclass
```

```
//=============================================================
// ENVIRONMENT
//=============================================================
class environment;
    generator  gen;
    driver     drv;
    monitor    mon;
    scoreboard scb;

    mailbox gen_drv, gen_scb, mon_scb;
    semaphore sem;
    event     done;

    function new(virtual apb_if vif);
        gen_drv = new();
        gen_scb = new();
        mon_scb = new();
        sem     = new(1);
        gen = new(gen_drv, gen_scb, done);
        drv = new(vif, gen_drv, sem);
        mon = new(vif, mon_scb);
        scb = new(gen_scb, mon_scb, done);
    endfunction

    task run();
        fork
            gen.run();
            drv.run();
            mon.run();
            scb.run();
        join_none
    endtask
endclass

//=============================================================
// TOP-LEVEL TESTBENCH (with VCD dumping)
//=============================================================
module tb;
    logic clk;
    logic rst_n;

    initial begin
        clk = 0;
        forever #5 clk = ~clk;
    end

    initial begin
        rst_n = 0;
        #10 rst_n = 1;
    end
```

```
    apb_if apb(clk, rst_n);
    apb_slave dut (
        .pclk    (clk),
        .presetn (rst_n),
        .psel    (apb.psel),
        .penable (apb.penable),
        .pwrite  (apb.pwrite),
        .paddr   (apb.paddr),
        .pwdata  (apb.pwdata),
        .prdata  (apb.prdata),
        .pready  (apb.pready),
        .pslverr (apb.pslverr)
    );

    environment env;
```

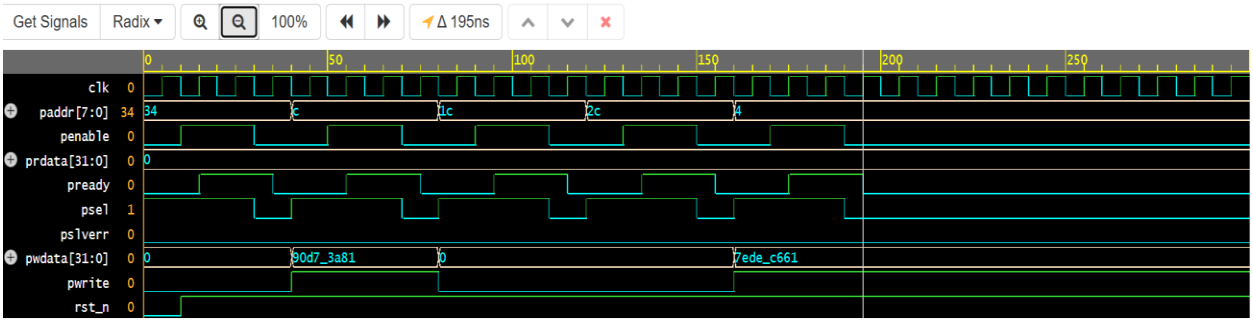
```
    initial begin
        $dumpfile("dump.vcd");
        $dumpvars(0, tb);

        env = new(apb);
        env.run();
        #300;
        $display("==================================");
        $display("TEST FINISHED");
        $display("==================================");
        $finish;
    end
endmodule
```

Output:

```
xcelium> source /xcelium25.03/tools/xcelium/files/xmsimrc
xcelium> run
[GENERATOR] ADDR=34 READ  DATA=0
[DRIVER] ADDR=34 READ  DATA=0
[MONITOR] ADDR=34 READ  DATA=0
SCOREBOARD: READ PASS (ADDR=34 DATA=0)
[GENERATOR] ADDR=c WRITE DATA=90d73a81
[DRIVER] ADDR=c WRITE DATA=90d73a81
[MONITOR] ADDR=c WRITE DATA=90d73a81
SCOREBOARD: WRITE PASS (ADDR=c DATA=90d73a81)
[GENERATOR] ADDR=1c READ  DATA=0
[DRIVER] ADDR=1c READ  DATA=0
[MONITOR] ADDR=1c READ  DATA=0
SCOREBOARD: READ PASS (ADDR=1c DATA=0)
```

```
[GENERATOR] ADDR=2c READ  DATA=0
[DRIVER] ADDR=2c READ  DATA=0
[MONITOR] ADDR=2c READ  DATA=0
SCOREBOARD: READ PASS (ADDR=2c DATA=0)
[GENERATOR] ADDR=4 WRITE DATA=7edec661
[DRIVER] ADDR=4 WRITE DATA=7edec661
[MONITOR] ADDR=4 WRITE DATA=7edec661
SCOREBOARD: WRITE PASS (ADDR=4 DATA=7edec661)
[GENERATOR] All transactions completed
================================
TEST FINISHED
================================
Simulation complete via $finish(1) at time 300 NS + 0
./testbench.sv:249          $finish;
xcelium> exit
```

Waveform:



Conclusion

The OOP-based verification environment successfully validated the APB slave design across all 5 randomized transactions, with the scoreboard reporting passes for every write and read operation. Waveform analysis confirmed proper APB protocol handshaking and correct data transfer. The use of mailboxes, semaphores, and events enabled seamless component synchronization, demonstrating the reusability and scalability of this layered testbench approach.